

# Technical Discussion of `netcenter`

Network Construction, Shortest Paths, the 1-Median, and the Absolute 1-Center

Technical note for `netcenter` v0.3.0

15 August 2026

## Abstract

`netcenter` solves three closely related facility-location problems on an undirected road network: a weighted network 1-median restricted to vertices, a vertex-restricted minimax 1-center, and an absolute 1-center whose facility may lie anywhere along an edge. The implementation combines GIS preprocessing, sparse shortest paths, and a vectorized exact edge sweep for the absolute center. This note formalizes the mathematical model, explains how each source module implements the model, describes the computational optimizations, and audits the differences between the current v0.3.0 code and the initial prototype.

The central conclusion is that *the facility-location algorithms have not been replaced*. The median still relies on Hakimi’s vertex-optimality result, and the absolute center is still obtained by minimizing the upper envelope of edgewise distance functions. Most changes since the prototype are validation, memory management, or exact-preserving reductions. The important substantive deviations are instead in *network construction*: the code no longer planarizes every geometric crossing by default, it recovers junctions from shared source vertices, it chooses the retained component by road length rather than node count, and it more strictly enforces metric coordinate systems. These choices can change the graph, and therefore can change the numerical location, without changing the mathematical location problem being solved on that graph.

## Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>What the package solves</b>                       | <b>3</b> |
| 1.1      | Weighted 1-median . . . . .                          | 3        |
| 1.2      | Vertex-restricted minimax center . . . . .           | 3        |
| 1.3      | Absolute 1-center . . . . .                          | 4        |
| <b>2</b> | <b>End-to-end computational pipeline</b>             | <b>4</b> |
| <b>3</b> | <b>Constructing the road graph</b>                   | <b>4</b> |
| 3.1      | Metric coordinate system . . . . .                   | 4        |
| 3.2      | Three topology modes . . . . .                       | 5        |
| 3.3      | Closed rings, slivers, and node compaction . . . . . | 5        |
| 3.4      | Parallel edges . . . . .                             | 5        |
| 3.5      | Connected component policy . . . . .                 | 6        |
| <b>4</b> | <b>Demand snapping and exact coalescing</b>          | <b>6</b> |
| <b>5</b> | <b>Shortest-path distance matrix</b>                 | <b>6</b> |
| 5.1      | Chunking and parallel assembly . . . . .             | 7        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>6</b>  | <b>The absolute-center edge sweep</b>                    | <b>7</b>  |
| 6.1       | Distance to demand from an interior edge point . . . . . | 7         |
| 6.2       | Upper envelope on one edge . . . . .                     | 7         |
| 6.3       | Safe lower-bound pruning . . . . .                       | 8         |
| 6.4       | Numerical exactness . . . . .                            | 8         |
| <b>7</b>  | <b>Pseudocode for the complete solution</b>              | <b>9</b>  |
| <b>8</b>  | <b>Complexity and practical bottlenecks</b>              | <b>10</b> |
| <b>9</b>  | <b>Did v0.3.0 deviate from the original algorithms?</b>  | <b>10</b> |
| 9.1       | Short answer . . . . .                                   | 10        |
| 9.2       | Detailed audit . . . . .                                 | 10        |
| 9.3       | The most important distinction . . . . .                 | 12        |
| <b>10</b> | <b>Scope and limitations</b>                             | <b>12</b> |
| <b>11</b> | <b>Verification strategy</b>                             | <b>13</b> |
| <b>12</b> | <b>Conclusion</b>                                        | <b>13</b> |

# 1 What the package solves

Let an undirected road network be a finite metric graph

$$G = (V, E, \ell),$$

where  $V = \{1, \dots, n\}$  is the vertex set,  $E$  is the set of road segments, and every edge  $e \in E$  has positive length  $\ell_e$ . The geometric realization of the graph, including the interior of every edge, is denoted by  $|G|$ . Shortest-path distance on this metric graph is  $d_G(\cdot, \cdot)$ .

The package takes a finite set of demand locations

$$Q = \{q_1, \dots, q_k\} \subseteq V$$

after demand points have been snapped to network vertices. Thus the facility may be continuous on the network for the absolute-center problem, but demand is *discrete vertex demand*. This distinction is essential: the package does not solve the variant in which every point of every road is itself a demand point.

For the selected demand vertices, define the demand-by-node distance matrix

$$D_{iv} = d_G(q_i, v), \quad D \in \mathbb{R}^{k \times n}.$$

The code in `distances.py` constructs this matrix; the optimization routines in `center.py` then work almost entirely with  $D$  and the edge list.

## 1.1 Weighted 1-median

Given nonnegative demand weights  $w_i$ , the weighted 1-median is

$$x_M \in \arg \min_{x \in |G|} \sum_{i=1}^k w_i d_G(q_i, x).$$

For vertex demand on a network, Hakimi's vertex-optimality theorem implies that an optimum can be chosen at a vertex [3]. Consequently `weighted_median()` evaluates

$$F(v) = \sum_{i=1}^k w_i D_{iv}$$

for every  $v \in V$  and returns a minimizing vertex. In matrix notation the weighted calculation is simply

$$(F(1), \dots, F(n)) = w^\top D.$$

If no weights are supplied, all  $w_i = 1$ .

## 1.2 Vertex-restricted minimax center

The second objective restricts the facility to  $V$  and minimizes the largest distance to the selected demand set:

$$x_V \in \arg \min_{v \in V} \max_{1 \leq i \leq k} D_{iv}.$$

The quantity

$$\text{ecc}_Q(v) = \max_i D_{iv}$$

is the eccentricity of  $v$  relative to the chosen demand set. When  $Q = V$  this is the classical graph-center or Jordan-center problem. When  $Q \subsetneq V$ , “Jordan center” is convenient shorthand in the code, but the precise object is a *demand-restricted vertex 1-center*.

### 1.3 Absolute 1-center

The absolute 1-center removes the vertex restriction on the facility:

$$x_A \in \arg \min_{x \in |G|} \max_{1 \leq i \leq k} d_G(q_i, x).$$

Hakimi showed that, unlike the median, an absolute center need not be a graph vertex [3]. The optimum may occur strictly inside a road segment. The main algorithmic work of **netcenter** is therefore an exact search of edge interiors after the shortest-path matrix is known.

## 2 End-to-end computational pipeline

The repository deliberately separates GIS semantics from mathematical optimization. The major modules have the following roles.

| Module                    | Responsibility                                                                                                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>graph.py</code>     | Reads linework, chooses a metre-based CRS, establishes junction topology, creates graph vertices and road segments, retains a connected component, and snaps external demand points to network nodes. |
| <code>topology.py</code>  | Converts endpoint arrays into a symmetric sparse adjacency matrix while preserving shortest parallel-edge cost.                                                                                       |
| <code>distances.py</code> | Computes shortest-path distances from demand nodes using SciPy's Dijkstra implementation, in bounded source chunks.                                                                                   |
| <code>center.py</code>    | Implements the weighted 1-median, demand-restricted vertex 1-center, lower-bound pruning, and the continuous edge sweep for the absolute 1-center.                                                    |
| <code>solve.py</code>     | Coalesces repeated snapped demands, orchestrates the distance and optimization stages, and converts abstract results back to map coordinates.                                                         |
| <code>cli.py</code>       | Exposes the pipeline to files and command-line options.                                                                                                                                               |

Conceptually, the computation is

$$\text{linework} \longrightarrow G \longrightarrow Q \longrightarrow D \longrightarrow \{x_M, x_V, x_A\}.$$

A wrong answer may therefore arise either because the optimization is wrong or because the constructed graph is not the road network the analyst intended. In applied GIS work, the second risk is often the larger one.

## 3 Constructing the road graph

### 3.1 Metric coordinate system

All edge lengths, snap-grid spacing, demand snap distances, and reported offsets are interpreted as metres. The builder therefore requires a projected, metre-based working CRS. Geographic input is reprojected to an inferred local UTM CRS unless the caller supplies an explicit target CRS. Already projected metre-based data are preserved rather than being reprojected unnecessarily. Projected data in other linear units are converted to a metre-based CRS when possible.

This is not part of the classical facility-location algorithm; it is a measurement precondition. Dijkstra and the location objectives are only as meaningful as the edge weights supplied to them.

### 3.2 Three topology modes

A central design question is whether two lines that cross in the plane are actually connected in the road network.

**Default: shared-source-vertex noding.** The current default does *not* create a junction at every geometric crossing. Instead, a coordinate location already used by at least two distinct input LineStrings is treated as source evidence of a junction. If that shared coordinate occurs in the interior of either line, the line is split there. This recovers common OSM-style T- and X-junctions while leaving an ordinary bridge/underpass crossing disconnected when the two ways share no source vertex.

Coordinate identity is determined on an integer quantization grid. For grid spacing  $s > 0$ , a coordinate  $p = (x, y)$  is mapped to

$$q_s(p) = (\text{round}(x/s), \text{round}(y/s)) \in \mathbb{Z}^2.$$

Locations with the same integer pair are treated as identical for node identity. This is deliberately described as a *grid spacing*, not as a Euclidean “within  $s$  metres” tolerance: two very close points can fall on opposite grid boundaries, while two points in one grid cell can differ along both axes.

**Strict endpoint-only mode.** If shared-vertex splitting is disabled, topology is inherited only from line endpoints. This is appropriate when the source has already been normalized so that every true junction is an endpoint of each incident segment.

**Optional full planar noding.** With `node=True`, the linework is unioned/noded so that geometric crossings become graph junctions. GEOS/Shapely provides the geometric operation. This mode is correct for a truly planar network, but it can invent false turns between grade-separated roads. Shapely explicitly describes full noding as adding all segment intersections to linework [6].

### 3.3 Closed rings, slivers, and node compaction

A closed road ring has coincident endpoints. If such a geometry were converted directly into an endpoint graph, it could become a self-loop and then disappear when self-loops are discarded. The builder therefore cuts closed and nearly-closed rings into ordinary segments before endpoint indexing.

After endpoint identities are assigned, zero-length/self-loop artifacts and segments below `min_length` are removed. Because this filtering can leave node identifiers no surviving edge uses, v0.3.0 compacts node IDs before the sparse graph is created. This prevents an unused row/column from appearing as an isolated connected component.

### 3.4 Parallel edges

The geometric edge list may contain more than one road segment joining the same pair of graph vertices. The sparse adjacency used for shortest paths must then store the *shortest* direct connection,

$$c_{uw} = \min_{e: e=(u,w)} \ell_e,$$

rather than the sum of duplicate sparse entries. The full geometric edge list is still retained for the absolute-center sweep, so a longer parallel road can remain a possible facility location even though endpoint-to-endpoint shortest paths use the shorter parallel connection.

### 3.5 Connected component policy

The default build retains one connected component so that all selected demands can reach all retained vertices. v0.3.0 chooses the component with the greatest total retained road length,

$$C^* = \arg \max_C \sum_{e \in E(C)} \ell_e,$$

rather than the component with the greatest number of vertices. This is a modeling/preprocessing policy, not a facility-location theorem. Analysts for whom islands or disconnected service territories are substantively important should solve those components separately.

## 4 Demand snapping and exact coalescing

External demand points are attached to their nearest *network node* using a k-d tree, an application of the multidimensional binary-search-tree idea of Bentley [1]. If  $z_i \in \mathbb{R}^2$  is an external point and  $p_v$  is a network-node coordinate, the snapped node is

$$q_i = \arg \min_{v \in V} \|z_i - p_v\|_2.$$

A maximum allowed snapping distance can be imposed so that highly simplified roads do not silently move demand by an implausible amount.

Repeated demand observations can snap to the same vertex. Before Dijkstra, `solve.py` coalesces those duplicates. This is mathematically exact. For the center objective,

$$\max\{d(q, x), d(q, x), \dots\} = d(q, x),$$

so multiplicity is irrelevant. For the median, all weights assigned to the same snapped vertex are summed:

$$\tilde{w}_q = \sum_{i: q_i = q} w_i.$$

Therefore the total weighted-distance objective is unchanged. This reduction can substantially decrease the number of Dijkstra source rows without changing any objective value.

## 5 Shortest-path distance matrix

`distances.py` computes

$$D_{iv} = d_G(q_i, v)$$

with Dijkstra's shortest-path algorithm [2]. The graph is required to be square, nonnegative, finite on stored edges, and symmetric. The symmetry check matters because the package calls SciPy in undirected mode and does not model one-way roads or asymmetric travel costs. SciPy exposes Dijkstra for both directed and undirected sparse graphs [5].

For  $k$  source demands,  $n$  vertices, and  $m$  sparse edges, a conventional binary-heap complexity statement is approximately

$$O(k(m + n \log n)),$$

although actual SciPy performance depends on implementation details and graph structure. The returned matrix itself requires

$$O(kn)$$

storage, which is often the practical memory bottleneck.

## 5.1 Chunking and parallel assembly

SciPy's Dijkstra output is produced in bounded groups of source vertices. If a chunk contains  $r$  sources, its temporary result has roughly  $rn$  floating-point cells. The code chooses  $r$  from `max_temp_mb` so that one float64 Dijkstra block remains within a target memory budget.

If the requested final dtype is float32, each completed block is cast before it is written into the final preallocated matrix. Parallel blocks are consumed as an ordered generator rather than accumulated and then concatenated. These are memory-management choices only: they do not change which shortest paths Dijkstra computes.

## 6 The absolute-center edge sweep

### 6.1 Distance to demand from an interior edge point

Consider an edge  $e = (u, w)$  of length  $L$ . Parameterize a point  $x(t)$  on the edge by its distance  $t \in [0, L]$  from  $u$ . For demand  $q_i$ , let

$$a_i = d_G(q_i, u) = D_{iu}, \quad b_i = d_G(q_i, w) = D_{iw}.$$

Any route from the interior point to the rest of the graph must first reach one of the endpoints, so

$$d_G(q_i, x(t)) = \min\{a_i + t, b_i + L - t\}. \quad (1)$$

This is a tent-shaped piecewise-linear function: the route through  $u$  rises with slope  $+1$ , while the route through  $w$  falls with slope  $-1$ .

The two alternatives are equal at

$$t_i^* = \frac{b_i - a_i + L}{2}. \quad (2)$$

Because the edge itself is a path of length  $L$  between  $u$  and  $w$ , shortest paths obey

$$|a_i - b_i| \leq L,$$

which implies  $t_i^* \in [0, L]$  in exact arithmetic.

### 6.2 Upper envelope on one edge

The edge eccentricity is

$$E_e(t) = \max_i d_G(q_i, x(t)).$$

Sort the breakpoints so that

$$t_{(1)}^* \leq t_{(2)}^* \leq \dots \leq t_{(k)}^*.$$

Between any two consecutive breakpoints, the set of demands using the falling branch and the set using the rising branch are fixed. On interval  $I_j$ , define

$$A_j = \max_{i: t_i^* \geq I_j} a_i, \quad B_j = \max_{i: t_i^* \leq I_j} b_i,$$

with the obvious empty-set treatment at the two outer intervals. The upper envelope on that interval reduces to

$$E_e(t) = \max\{A_j + t, B_j + L - t\}. \quad (3)$$

The first term is increasing and the second decreasing, so their maximum is a V-shaped convex function on the interval. The unconstrained crossing is

$$\hat{t}_j = \frac{B_j + L - A_j}{2}.$$

The interval minimizer is therefore

$$t_j = \text{clip}(\hat{t}_j; \underline{t}_j, \bar{t}_j), \quad (4)$$

where clipping moves an out-of-interval crossing to the nearest endpoint. Evaluating (3) at all  $k + 1$  intervals and taking the best gives the exact edge minimum in exact arithmetic.

The implementation computes  $A_j$  with suffix running maxima and  $B_j$  with prefix running maxima. Thus, after sorting the  $k$  breakpoints, the remaining work is linear in  $k$ . The cost per swept edge is

$$O(k \log k).$$

This is a direct computational realization of the classical absolute-center structure initiated by Hakimi and developed algorithmically in subsequent network-location work, including Kariv and Hakimi [3, 4]. The package does not implement later linear-time/parametric-search refinements.

### 6.3 Safe lower-bound pruning

Sweeping every edge would have worst-case cost  $O(mk \log k)$ . Before sorting, `absolute_center()` obtains a valid incumbent radius from the best vertex,

$$R_V = \min_{v \in V} \max_i D_{iv}.$$

For any point  $x(t)$  on edge  $(u, w)$  and any demand  $i$ ,

$$d_G(q_i, x(t)) = \min\{a_i + t, b_i + L - t\} \geq \min\{a_i, b_i\}.$$

Therefore every point on the edge satisfies

$$E_e(t) \geq LB_e := \max_i \min\{D_{iu}, D_{iw}\}. \quad (5)$$

If  $LB_e$  is already no better than the incumbent (allowing a conservative floating-point margin), the edge cannot materially improve the current answer and need not be sorted. This is exact branch-and-bound logic in real arithmetic. It is an implementation acceleration, not a different center objective.

### 6.4 Numerical exactness

The mathematical sweep is exact for a supplied shortest-path metric. The software necessarily adds floating-point policy:

- breakpoints are defensively clipped to  $[0, L]$  against tiny numerical violations of the triangle inequality;
- a scale-aware pruning margin prevents float32 rounding from discarding a borderline candidate edge;
- improvements smaller than the configured numerical tolerance can be reported as tied with the incumbent vertex.

Accordingly, “exact” should be read as *algorithmically exact modulo the explicit floating-point tolerance and the precision of the stored distance matrix*. With the default float64 matrix and the default tolerance, this numerical qualification is normally negligible. A float32 run is an optional memory-saving approximation to the original float64 metric, even though the search is internally consistent with the rounded matrix.

## 7 Pseudocode for the complete solution

The following pseudocode mirrors the current architecture without Python-specific details.

```

BUILD-NETWORK(linework, grid s, topology_mode):
    transform linework to a projected metre-based CRS
    remove empty/non-line geometries

    if topology_mode == PLANAR:
        split at every geometric crossing
    else if topology_mode == SHARED_VERTEX:
        split at interior coordinates shared by distinct source lines

    split closed or nearly closed rings
    quantize segment endpoints on grid s and assign node IDs
    remove self-loops and slivers
    compact unused node IDs

    create sparse undirected adjacency:
        for parallel node pairs, retain minimum length for shortest paths

    optionally retain the connected component with greatest road length
    return Network(nodes, geometric_edges, sparse_adjacency)

SOLVE(Network G, snapped demands Q, weights w):
    coalesce repeated demand nodes
        - duplicates vanish for minimax objectives
        - duplicate median weights are summed

    D <- multi-source Dijkstra distances from Q to all graph nodes

    median <- argmin_v sum_i w_i D[i,v]
    vertex_center <- argmin_v max_i D[i,v]

    incumbent <- vertex_center
    for each geometric edge e=(u,w,L):
        LB[e] <- max_i min(D[i,u], D[i,w])
    candidates <- edges whose LB can improve incumbent

    for each candidate edge e:
        a_i <- D[i,u]
        b_i <- D[i,w]
        t_i* <- (b_i - a_i + L)/2
        sort demands by t_i*
        build suffix maxima of a_i and prefix maxima of b_i
        analytically minimize the V-envelope on each breakpoint interval
        keep the best point on e

```

```
absolute_center <- better of incumbent and all candidate-edge minima
return median, vertex_center, absolute_center
```

## 8 Complexity and practical bottlenecks

Let  $n = |V|$ ,  $m = |E|$ , and let  $k$  denote the number of *unique snapped* demand nodes after coalescing.

| Stage                      | Indicative cost                       | Practical note                                                |
|----------------------------|---------------------------------------|---------------------------------------------------------------|
| Demand nearest-node lookup | about $O(k \log n)$ per query set     | k-d-tree behavior depends on dimension/data distribution.     |
| Dijkstra distance matrix   | $O(k(m+n \log n))$ conventional bound | Usually the dominant computation; matrix storage is $O(kn)$ . |
| Weighted 1-median          | $O(kn)$                               | One weighted column reduction.                                |
| Vertex 1-center            | $O(kn)$                               | One columnwise maximum plus minimum.                          |
| Edge lower bounds          | $O(km)$ worst case                    | Vectorized and blocked.                                       |
| Absolute-center sweep      | $O(mk \log k)$ worst case             | In practice only unpruned edges are sorted.                   |

The package is therefore not primarily CPU-bound by the final edge sweep on many realistic instances. The  $k \times n$  shortest-path matrix is often the larger computational and memory constraint. This is why duplicate-demand coalescing, chunking, and careful dtype selection matter.

## 9 Did v0.3.0 deviate from the original algorithms?

### 9.1 Short answer

**No, not in the core facility-location mathematics.** The current code still solves the same weighted 1-median, demand-restricted vertex 1-center, and absolute 1-center with vertex demand. The absolute-center implementation still uses the same edgewise breakpoint/upper-envelope sweep as the initial prototype.

**Yes, in the end-to-end graph-construction behavior.** Several deliberate changes alter how GIS linework becomes the graph on which those unchanged algorithms operate. Those changes can legitimately move the final centers because they change  $G$ , not because they change the optimization objective.

### 9.2 Detailed audit

| Aspect            | Initial prototype                                           | v0.3.0                                     | Algorithmic deviation? |
|-------------------|-------------------------------------------------------------|--------------------------------------------|------------------------|
| Weighted 1-median | Minimize weighted total shortest-path distance at vertices. | Same objective and same vertex evaluation. | <b>No.</b>             |

| Aspect                     | Initial prototype                                                                             | v0.3.0                                                                                                                          | Algorithmic deviation?                                                       |
|----------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Vertex minimax center      | Minimize maximum demand distance over vertices.                                               | Same objective and reduction.                                                                                                   | <b>No.</b>                                                                   |
| Absolute 1-center          | Edge-interior facility location using sorted tent breakpoints and prefix/suffix maxima.       | Same breakpoint/envelope sweep.                                                                                                 | <b>No.</b>                                                                   |
| Shortest paths             | Dijkstra on a sparse undirected graph.                                                        | Same Dijkstra stage, now validated, chunked, and stream-assembled.                                                              | <b>No.</b>                                                                   |
| Lower-bound pruning        | $\max_i \min(D_{iu}, D_{iw})$ used to avoid hopeless edges.                                   | Same mathematical bound with a more conservative numerical margin.                                                              | <b>No;</b> safer implementation.                                             |
| Vectorization / blocking   | Batched NumPy sweep with a memory cell budget.                                                | Retained, with stricter parameter validation.                                                                                   | <b>No.</b>                                                                   |
| Demand duplicates          | Repeated snapped demand could lead to repeated Dijkstra source rows.                          | Unique snapped nodes are solved once; median weights/counts are aggregated.                                                     | <b>No;</b> exact reduction.                                                  |
| Default crossing treatment | Original prototype defaulted to full planar noding (every 2-D crossing treated as connected). | Default avoids invented crossings and instead splits at vertices shared by distinct source lines; full planar noding is opt-in. | <b>Yes, preprocessing/model semantics.</b>                                   |
| Shared-vertex junctions    | Not a distinct topology mode in the earliest prototype.                                       | Recovers T- and X-junctions from source vertices without connecting mere bridge crossings.                                      | <b>Yes, graph construction.</b>                                              |
| Connected component choice | Largest component by node count.                                                              | Component with greatest total road length.                                                                                      | <b>Yes, preprocessing policy.</b>                                            |
| CRS handling               | Earlier versions could make weaker assumptions about projected units.                         | Explicitly requires/constructs a metre-based projected CRS and preserves valid metric projected input.                          | <b>Yes, measurement policy;</b> objective unchanged.                         |
| Parallel-edge construction | Early sparse conversion could sum duplicate entries.                                          | Minimum direct edge length is stored for each node pair.                                                                        | <b>Bug fix,</b> restoring intended shortest-path graph.                      |
| Closed rings               | Could be lost as self-loops in early construction.                                            | Rings are split before endpoint graph creation.                                                                                 | <b>Bug fix,</b> restoring intended geometry.                                 |
| Unused node IDs            | Filtering could leave orphan CSR rows/columns.                                                | Nodes are compacted after filtering.                                                                                            | <b>Bug fix.</b>                                                              |
| Numerical precision        | Primarily float64-oriented.                                                                   | float64 remains available/default in the Python API; float32 storage is optional with conservative pruning tolerance.           | <b>Optional numerical approximation</b> only when lower precision is chosen. |

| Aspect            | Initial prototype                              | v0.3.0                                                                                | Algorithmic deviation?       |
|-------------------|------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------|
| Parallel defaults | More aggressive worker defaults in early code. | One worker is the conservative default; process/thread backends are explicit choices. | <b>No</b> ; scheduling only. |

### 9.3 The most important distinction

There are two different senses of “same algorithm”:

1. **Same optimization problem and mathematical method.** In this sense v0.3.0 is highly faithful to the original. The objective functions, shortest-path metric, and edge sweep are unchanged in substance.
2. **Same end-to-end mapping from a GIS file to coordinates.** In this sense v0.3.0 is deliberately *not* identical. A bridge crossing, a shared OSM vertex, a disconnected fragment, a non-metre CRS, or a filtered sliver may produce a different graph than in the earliest version. A mathematically identical center algorithm applied to a different graph can of course return a different center.

This distinction is desirable. The purpose of the revisions was not to invent a new facility-location algorithm, but to ensure that the classical algorithm is applied to a graph that more faithfully encodes the analyst’s intended road network.

## 10 Scope and limitations

The current implementation is deliberately narrower than the full network location literature.

- **Undirected network only.** One-way roads, turn restrictions, and asymmetric time costs require a directed-state representation and a different interior-edge treatment.
- **Unweighted minimax center.** Demand weights affect the median but not the center. A weighted minimax objective changes the slopes of the edgewise functions and cannot simply reuse the present suffix/prefix structure unchanged.
- **Vertex demand.** The facility may lie inside an edge, but the demand set is finite and snapped to vertices. This is not the all-points continuous-network center.
- **One facility.** The package solves  $p = 1$ . General  $p$ -center and  $p$ -median problems have substantially different computational difficulty; Kariv and Hakimi’s 1979 papers are foundational for that broader setting [4].
- **Node snapping rather than edge projection.** External demand is assigned to the nearest node. On heavily simplified road data this can be a strong modeling assumption, which is why snap distances should be inspected and bounded.
- **Geometry does not encode all traffic rules.** Shared source vertices are treated as evidence of connectivity, but the builder does not interpret OSM-style bridge, tunnel, layer, one-way, or turn-restriction tags.

## 11 Verification strategy

The test suite separates mathematical correctness from GIS correctness as much as possible. The continuous edge sweep is checked against independently sampled brute-force searches on synthetic networks; hand-computable cases test median and center objectives; regression tests cover parallel edges, rings, shared-vertex T- and X-junctions, grade-separated non-junctions, disconnected components, invalid indices, asymmetric CSR input, and chunked shortest paths.

Brute-force comparison is especially useful because it does not share the breakpoint logic of the sweep. Nevertheless, it validates the optimizer only *conditional on the supplied graph and distance matrix*. No numerical test can determine whether a bridge in the source data was meant to connect to the road below. Network semantics remain an input-data responsibility.

## 12 Conclusion

`netcenter` is best understood as a classical network-location solver surrounded by unusually explicit GIS and numerical safeguards. The mathematical core is compact:

1. build the shortest-path metric from selected demand vertices;
2. evaluate the weighted 1-median and the vertex minimax center directly from  $D$ ;
3. use the vertex center as an incumbent;
4. discard edges that cannot beat it using (5);
5. on each remaining edge, sort the route-switch breakpoints and minimize the piecewise V-shaped upper envelope exactly, modulo floating-point tolerance.

Relative to the initial prototype, v0.3.0 does *not* constitute a new absolute-center algorithm. Its principal advances are engineering and model fidelity: safer topology, more defensible metric units, exact duplicate-demand reduction, bounded-memory shortest paths, stricter validation, and regression coverage of graph-construction failures. The only changes likely to move a result materially are precisely those that change the graph being represented or deliberately reduce numerical precision.

## References

- [1] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [2] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [3] S. L. Hakimi. Optimum locations of switching centers and the absolute centers and medians of a graph. *Operations Research*, 12(3):450–459, 1964. doi:10.1287/opre.12.3.450.
- [4] O. Kariv and S. L. Hakimi. An algorithmic approach to network location problems. I: The  $p$ -centers. *SIAM Journal on Applied Mathematics*, 37(3):513–538, 1979. doi:10.1137/0137040.
- [5] SciPy developers. `scipy.sparse.csgraph.dijkstra` documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.dijkstra.html>, accessed 15 August 2026.

- [6] Shapely developers. `shapely.node` documentation, Shapely 2.1.2. <https://shapely.readthedocs.io/en/2.1.2/reference/shapely.node.html>, accessed 15 August 2026.